

OpenMP Multi-Core Scaling in Python: Comprehensive Lab Report

Student Name: Akbota Kurman

Student ID: 230103276_01N_03P 01.10.2026

Hardware / CPU: 4 Hardware Cores Detected

1. Executive Summary & Combined Result Scorecard (Table 1)

Benchmark Challenge	Active Threads / Mode	Array Size / Samples	Execution Time (s)	Speedup Factor	Parallel Efficiency (%)	Throughput / Rate
Challenge 1: Monte Carlo	1 Thread (Baseline)	10,000,000	1.4250 s	1.00x	100.0%	N/A
Challenge 1: Monte Carlo	2 Threads	10,000,000	1.3739 s	1.04x	51.9%	N/A
Challenge 1: Monte Carlo	4 Threads (Max Cores)	10,000,000	1.4084 s	1.01x	25.3%	N/A
Challenge 2: Mandelbrot (Rows)	2 Threads / Max Threads	200×200, 100 max iter	0.1184 s	N/A	N/A	N/A
Challenge 2: Mandelbrot (Cols)	2 Threads / Max Threads	200×200, 100 max iter	0.1781 s	N/A	N/A	N/A
Challenge 3: Heat Stencil	1 Thread (float64)	100×100×10	0.1865 s	1.00x	100.0%	0.54 MCells/s
Challenge 3: Heat Stencil	2 Threads (float64)	100×100×10	0.1515 s	1.23x	61.5%	0.66 MCells/s
Challenge 3: Heat Stencil	1 Thread (float32)	100×100×10	0.1618 s	1.00x	100.0%	0.62 MCells/s
Challenge 3: Heat Stencil	2 Threads (float32)	100×100×10	0.1932 s	0.84x	41.9%	0.52 MCells/s

2. Challenge 1: The Core Speedometer & Amdahl's Law

Technical Questions & Analysis

- A. Recorded Timings:

- Baseline Single-Thread Runtime (T1): **1.4250 s**
 - Maximum Multi-Thread Runtime (T4): **1.4084 s**
- **B. Why is the warm-up call (`_ = monte_carlo_pi(10_000)`) strictly required before recording benchmark timings?**
Numba utilizes Just-In-Time (JIT) compilation. During the initial invocation, the JIT engine compiles Python bytecode into machine instructions via LLVM. The warm-up call absorbs this compilation overhead so that timing measurements reflect pure, bare-metal parallel execution speed.
 - **C. Did parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors explain the degradation?**
No, parallel efficiency dropped to **25.3%**. Key factors include:
 - **PRNG Lock Contention:** Calling `np.random.uniform()` inside a `prange` loop forces thread synchronization around Numba's internal pseudo-random number generator state.
 - **Low Compute Density:** The arithmetic workload per sample ($x^2+y^2 \leq 1.0$) is minimal compared to thread management overhead.
 - **Thread Synchronisation Overhead:** Managing thread barriers across 4 cores consumes resources, negating parallel gains.
 - **D. Why does `inside_circle += 1` inside `prange` not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?**
Numba automatically detects accumulation operators (`+=`) inside parallel loops and converts them into an **OpenMP Reduction** construct (equivalent to `omp parallel for reduction(+:inside_circle)`). Each thread maintains a private local counter, which is safely aggregated into the global variable upon loop completion to avoid race conditions.

3. Challenge 2: Load Imbalance & Dynamic Scheduling

Technical Questions & Analysis

- **A. Compare `t_rows` vs `t_cols`. Which axis decomposition completed faster on your machine and why?**
Row-parallel decomposition completed faster (**0.1184 s** vs **0.1781 s** for column-parallel). Row-major traversal aligns directly with memory layout, optimizing memory access patterns.
- **B. Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]`.**
NumPy allocates multi-dimensional arrays in C-contiguous order (row-major). When threads write sequentially across rows (`img[r, c]`), adjacent elements share CPU cache

lines, maximizing cache hits and write efficiency. Writing down columns (`img[c, r]`) causes strided memory access, frequent cache misses, and false sharing across cores.

- **C. Load Imbalance Analysis:**

Complex plane points inside the Mandelbrot set require up to `max_iter` calculations, whereas exterior points escape in 2–3 iterations. Under static scheduling, a thread assigned to central rows works far longer than threads assigned to outer rows, leaving cores idle. OpenMP dynamic scheduling resolves this by breaking iterations into small chunks and assigning them to threads dynamically as they become available.

- **D. Output Artifact:**

The generated visualization has been successfully exported and saved as `mandelbrot_output.png`.

4. Challenge 3: Stencil Computation & Memory Bandwidth

Technical Questions & Analysis

- **A. Performance & Throughput Observations:**

For the 100×100 grid over 10 simulation steps, single-threaded execution completed in **0.1865 s** (`float64`) with a peak throughput of **0.66 MCells/s** under 2 threads.

- **B. Array Precision Impact (`float64` vs. `float32`):**

Switching single-threaded execution from `float64` to `float32` reduced execution time from **0.1865 s** to **0.1618 s** and increased throughput from **0.54 MCells/s** to **0.62 MCells/s**. Stencil computations are memory-bandwidth bound. Halving data precision from 64 bits to 32 bits reduces data volume per timestep by 50%, speeding up RAM transfers.

- **C. Memory Roofline Model Analysis:**

Once memory bus bandwidth is saturated, adding more CPU cores yields diminishing or negative returns (e.g., `float32` time increased from 0.1618 s to 0.1932 s with 2 threads). The Roofline Model demonstrates that when an algorithm is memory-bound, additional computing power cannot increase performance because threads spend most cycles waiting for memory access.